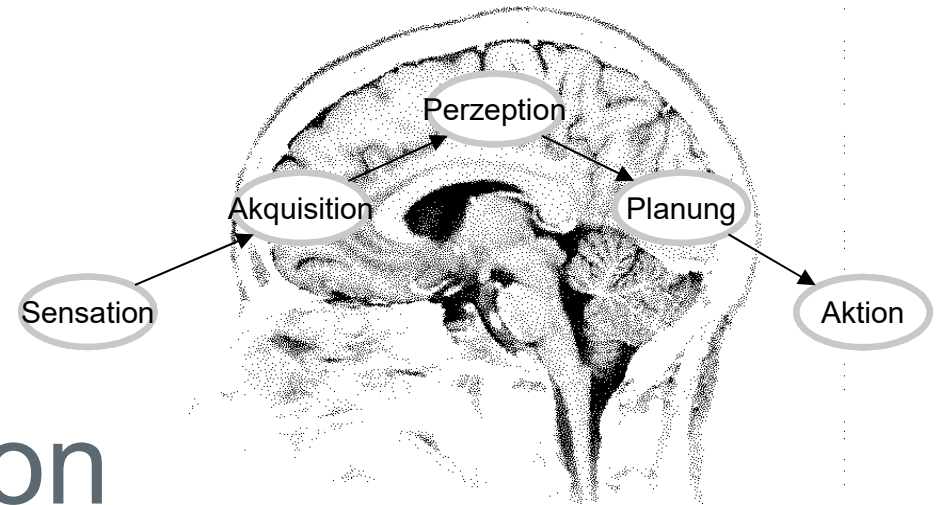
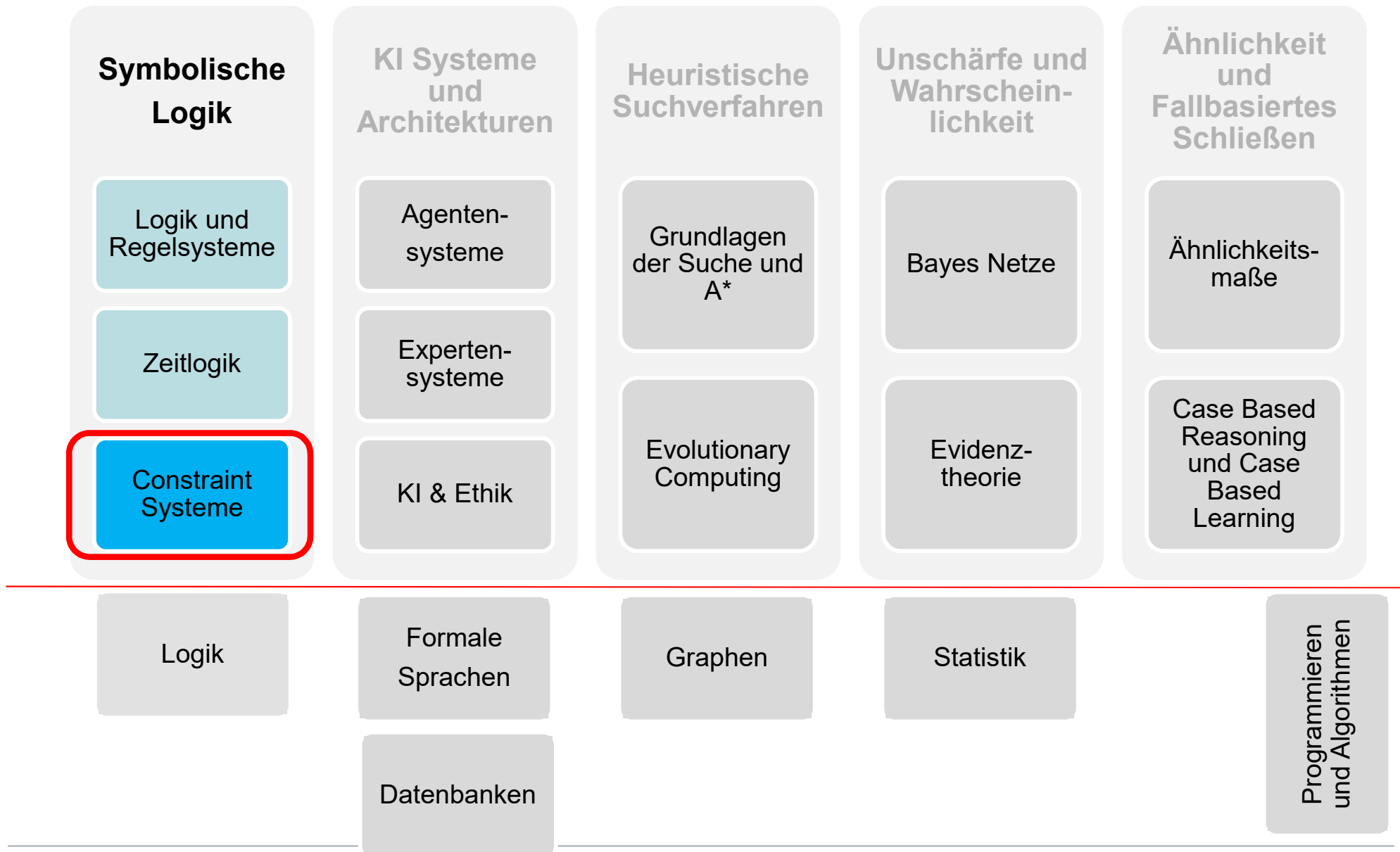




Constraint Satisfaction Problems

... eine Einführung



Constraint Satisfaction ... the basic idea

X – Set of Variables
D – Set of Domains
C – Set of Constraints

	8	2	4		1	3		
							1	
4	1	5		6		9	7	2
8	3	7			2			4
	9		3	4	5		8	
			7		6			3
6		4		3				1
	2							
			5		4	8	3	

numbers 1-9 once per row

unique 1-9 in a block

numbers 1-9 once per column

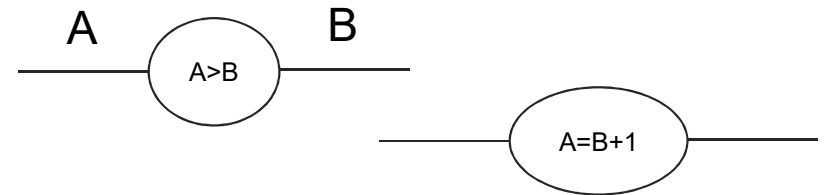
$C = (\{X_1, X_2, X_3, X_4, X_5, X_6, X_7, X_8, X_9\}, X_i \text{ unique})$

$D_i = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Constraint Networks

Simple approach

- Relations as nodes
- Variables as edges



Formally: a constraint is an n -ary relation, interpreted in a defined Universe

Important features of constraints:

$n-1$ arguments in an n -ary relation determine the n -th argument

Definition:

- (a) $r(x_1, \dots, x_n)$ is functional for the i -th argument, if all assignments to the other variables lead to exactly one value of x_i to fulfill the relation
 - (b) An n -ary relation is called fully functional, if it is functional for each argument.
-

Example: "Celsius-to-Fahrenheit"

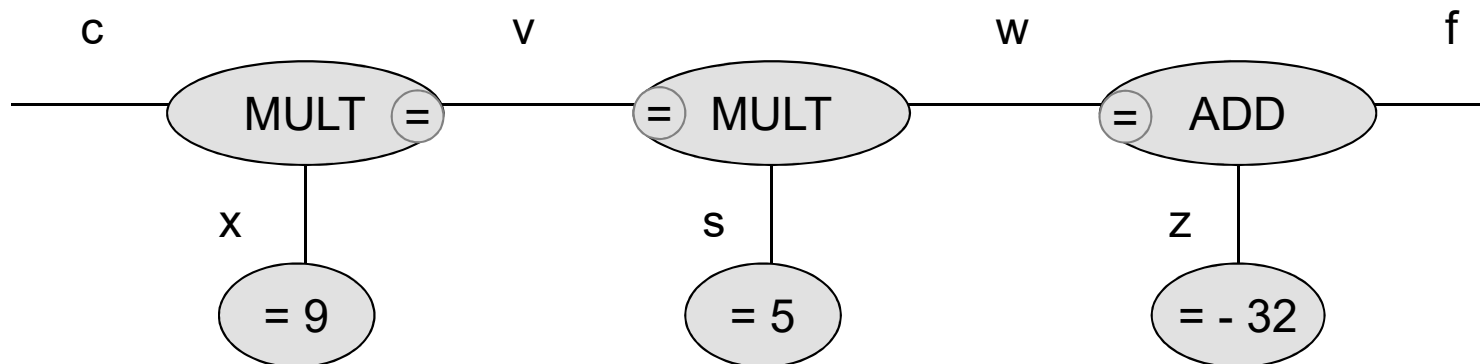
Celsius to Fahrenheit and vice versa

$$9 \cdot c = 5 \cdot (f - 32)$$

As a logical formula

$$\text{MULT}(x, c, v) \wedge x = 9 \wedge \text{ADD}(f, z, w) \wedge z = -32 \wedge \text{MULT}(w, s, v) \wedge s = 5$$

And in a graphical description



predicate definitions

$$\text{ADD}(x, y, z) = \{ (x, y, z) \mid x + y = z \}$$

$$\text{MULT}(x, y, z) = \{ (x, y, z) \mid x \cdot y = z \}$$

Example: "Celsius-to-Fahrenheit"

$$9 \cdot c = 5 \cdot (f - 32)$$

„Naive“ representation using PROLOG:

```
add(X,Y,Z):-var(Z),not(var(X)),not(var(Y)), Z is X + Y.  
add(X,Y,Z):-var(X),not(var(Y)),not(var(Z)), X is Z - Y.  
add(X,Y,Z):-var(Y),not(var(X)),not(var(Z)), Y is Z - X.
```

```
mult(X,Y,Z):-var(Z),not(var(X)),not(var(Y)), Z is X * Y.  
mult(X,Y,Z):-var(X),not(var(Y)),not(var(Z)), X is Z / Y.  
mult(X,Y,Z):-var(Y),not(var(X)),not(var(Z)), Y is Z / X.
```

```
c2f(C,F):-X = 9, Z = -32, S = 5, mult(X,C,V),mult(W,S,V),add(F,Z,W).  
c2f(C,F):-X = 9, Z = -32, S = 5, add(F,Z,W),mult(W,S,V),mult(X,C,V).
```

```
?- c2f(40,X).
```

```
X = 104
```

What is 40 C in F ?

```
?- c2f(X,0).
```

```
X = -17.7778
```

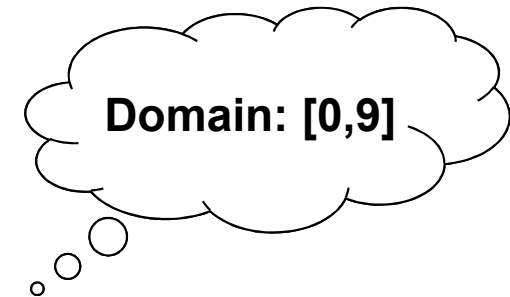
What is 0 F in C?

Example: “crypto-arithmetic problems”

... from literature

$$\begin{array}{r} \text{SEND} \\ + \text{MORE} \\ \hline \text{MONEY} \end{array}$$

$$\begin{array}{r} \text{TWO} \\ + \text{TWO} \\ \hline \text{FOUR} \end{array}$$



... as constraints

c1 $D + E = Y + 10 \cdot X_1$  helper variable for overflow

c2 $N + R + X_1 = E + 10 \cdot X_2$

c3 $E + O + X_2 = N + 10 \cdot X_3$

c4 $S + M + X_3 = O + 10 \cdot M$



How would you draw a constraint graph?

Constraint graphs

... for non-binary constraints

Definition (Constraint / Primal Graph for general CSPs):

Let $CSP = (X, D, C)$ be a constraint satisfaction problem, where

$X = \{X_1, \dots, X_n\}$ are the variables,

D_i are their domains, and

$C = \{C_1, \dots, C_m\}$ are the constraints.

The **Constraint Graph** (or **Primal Graph**) is an undirected graph $G = (V, E)$ such that:

- " $V = X$ (one node per variable),"
- " $(X_i, X_j) \in E$ iff there exists a constraint $C_k \in C$ that involves both X_i and X_j ."

In other words, **all variables appearing in the same constraint form a clique** in the primal graph.

Constraint graphs

... for non-binary constraints

Since the primal graph can become very dense when constraints involve many variables, other graph structures are used to represent non-binary CSPs more efficiently.

(a) Dual Graph

- **Nodes:** constraints
- **Edges:** connect two constraints if they **share at least one variable**

(b) Incidence Graph (Bipartite Graph)

- A **bipartite** representation with:
 - one set of nodes = variables
 - the other set = constraints
- An edge connects variable X_i to constraint C_j if and only if X_i is involved in C_j .

Constraint graphs

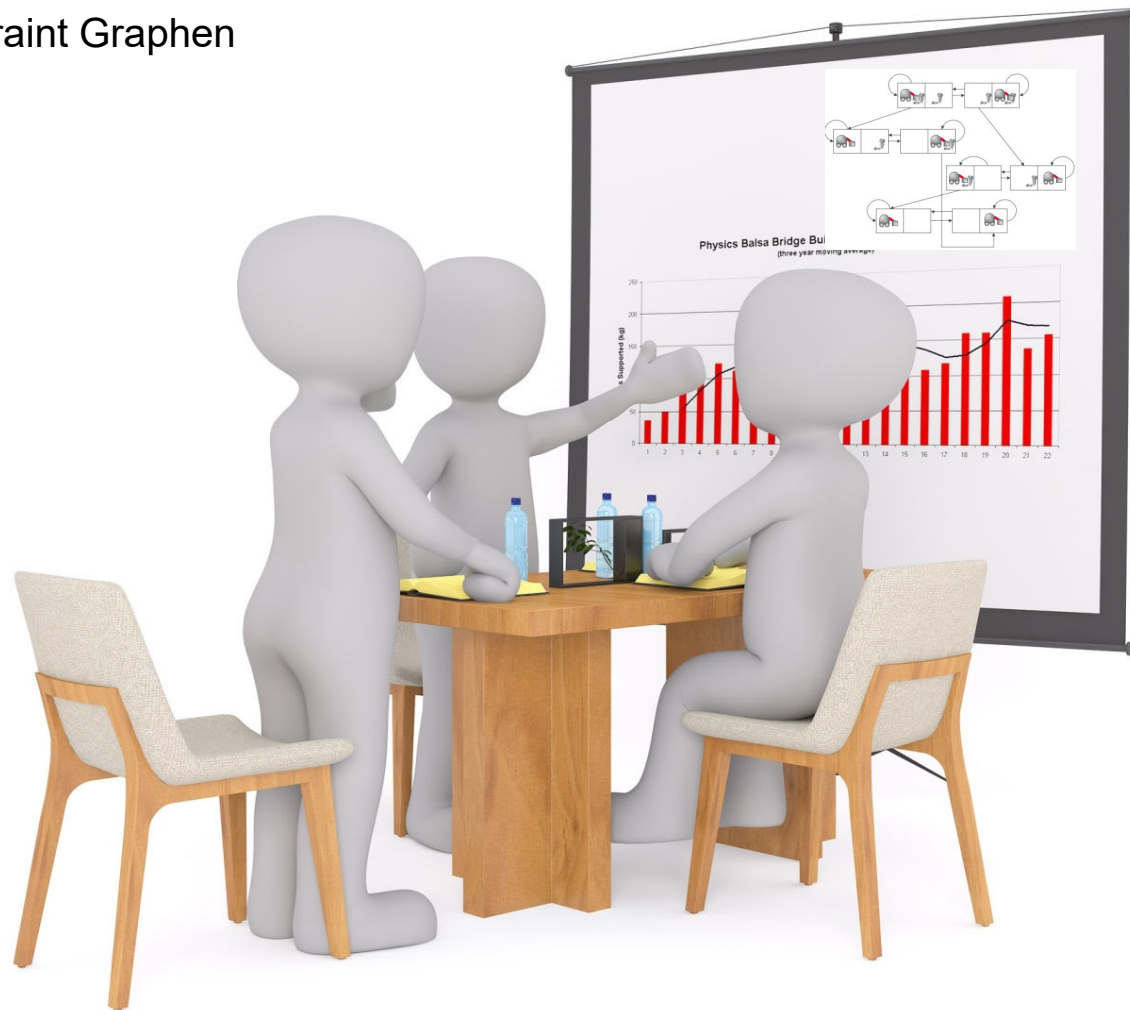
... an overview

Graph Type	Nodes	Edges connect...	Purpose
Constraint Graph (Primal)	Variables	Variables that appear together in a constraint	Shows variable–variable relationships
Dual Graph	Constraints	Constraints sharing at least one variable	Shows constraint–constraint relationships
Incidence Graph	Variables + Constraints	Variables to the constraints they appear in	Full bipartite structure

Eine kleine Übung

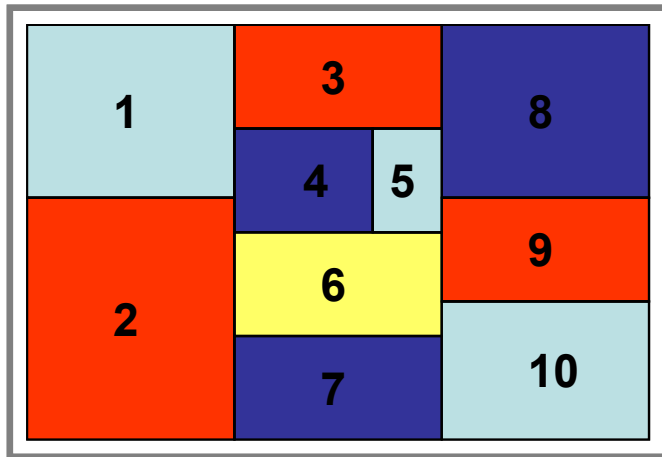
Erstellen Sie einen Constraint Graphen
für das folgende Problem

$$\begin{array}{r} \text{TWO} \\ + \text{TWO} \\ \hline \text{FOUR} \end{array}$$

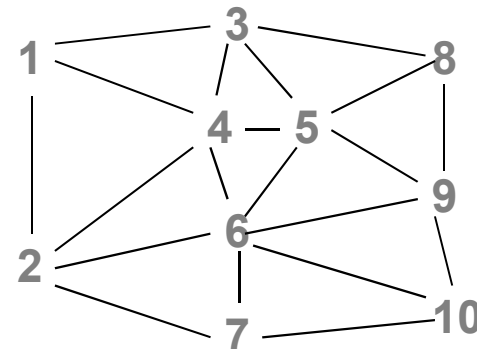


Beispiele für Constraint-Netze: "Graphenfärbung"

Geben Sie den Flächen eine Farbe. Bedingung: Es gibt nur 4 Farben und es dürfen keine zwei Flächen mit gleicher Farbe aneinander grenzen.



Darstellungsform als (primal) Constraint Graph:



1	rot	grün	blau	gelb
2	rot	grün	blau	gelb
3	rot	grün	blau	gelb
4	rot	grün	blau	gelb
5	rot	grün	blau	gelb
6	rot	grün	blau	gelb
7	rot	grün	blau	gelb
8	rot	grün	blau	gelb
9	rot	grün	blau	gelb
10	rot	grün	blau	gelb

Strategie:

Welche Farbe wird gewählt?

Welcher Knoten wird gewählt?

Constraint Netze - Inferenz

(a) Inferenz zur Erzeugung neuen Wissens

- Angeben von Werten für einzelne Variablen
- Ermitteln von Werten, die durch Relationen bestimmt werden
- Ausbreiten über das ganze Netz



Constraint-Propagation

(b) Inferenz zur Konsistenz-Prüfung

- Vorgehensweise wie oben.
- Versuch der Fortsetzung über das ganze Netz

Resultat: Eingaben sind konsistent, wenn dies möglich ist, sonst erhält man einen Widerspruch.

Constraint Netze - Inferenz

Zwei Vorgehensweisen der Constraint-Propagation

- (a) konstruktiv Ausgehend von einer Menge fester Werte (Anfangsbelegung) werden andere Werte berechnet, oder bei Mehrdeutigkeit als „Versuch“ gesetzt (**Backtracking!**).
- (b) destruktiv Ausgehend von der Menge aller Möglichkeiten wird der Lösungsbereich für die Variablen des Netzes sukzessive eingeschränkt.

Problem: Netze mit nicht-funktionalen Relationen

Notwendig ist eine große Zahl von funktionalen Relationen und eine Einschränkung der nicht-funktionalen auf Variablen mit kleinem, erlaubten Wertebereich.

→ *Revisionsmechanismen*

Constraint Netze - Inferenz

Heuristiken

Minimieren/Reduzieren des Verzweigungsgrades

- Die Variable mit dem kleinsten (verbleibenden) Wertebereich
- Die Variable welche in den meisten Constraints vorkommt

Erhalten der Entscheidungsfreiheit

- Die Variable wird auf einen Wert gesetzt, so dass möglichst wenige Lösungsmöglichkeiten dadurch ausgeschlossen werden
-

PRAXIS: Grundlagen in Python

```
from constraint import Problem

# Create a new CSP problem instance
problem = Problem()

# Step 1: Define variables and their domains
problem.addVariable('X1', [1, 2, 3])
problem.addVariable('X2', [2, 3])
problem.addVariable('X3', [1, 3])

# Step 2: Add constraints
# Unary constraint: X1 cannot be 3
problem.addConstraint(lambda x1: x1 != 3, ['X1'])

# Binary constraint: X1 < X2
problem.addConstraint(lambda x1, x2: x1 < x2, ['X1', 'X2'])

# Binary constraint: X2 != X3
problem.addConstraint(lambda x2, x3: x2 != x3, ['X2', 'X3'])

# Step 3: Retrieve and print all valid solutions
solutions = problem.getSolutions()
print(f"Found {len(solutions)} solutions:")
for s in solutions:
    print(s)
```

